

GPU Data Center Load-Following Demand Response: A Population-Based Simulation Framework with Heterogeneous Workloads

Laura Moreno
Cornell Tech

May 13th, 2026

Abstract

As AI workloads rapidly expand global data center electricity consumption, GPU clusters represent an underutilized source of demand-side flexibility for grid operators. This paper develops a population-based simulation framework that characterizes the power flexibility envelope of a heterogeneous GPU data center operating as a demand-response resource, subject to workload scheduling and hardware constraints. The framework tracks four LLM job types, namely prefill, decode, fine-tuning, and training, each with distinct execution time, deadline slack, and memory requirements, across three GPU memory classes. Control is implemented via a myopic single-slot linear program (LP) that allocates GPU compute frequencies through DVFS to minimize deviation from a prescribed power target. A λ -sweep experiment and per-class transition matrix analysis identify the stability boundary λ^* and reveal that the bilateral control window is empty under this policy: no arrival rate simultaneously achieves a bounded queue and full power-target tracking. This structural finding motivates the extension to Model Predictive Control (MPC), which is expected to close this gap by optimizing jointly over a planning horizon.

1 Introduction

The explosive growth of AI workloads has made GPU data centers among the fastest-growing electricity consumers globally. Unlike fixed industrial loads, AI compute clusters have an intrinsic flexibility where jobs can be throttled, deferred, or migrated without violating service-level agreements, and this could allow data centers to participate in load-following, frequency regulation, and capacity reserve programs. A central challenge is figuring out the marginal value of flexible power at a given operating point: the price at which a data center can credibly offer reserve capacity as a function of workload intensity, hardware configuration, and scheduling policy.

Most existing work models the compute load as a single aggregate resource, ignoring internal workload structure. In practice, an AI data center runs concurrent job types with fundamentally different profiles: prefill is compute-bound and latency-critical; decode is memory-bound and deferrable; training is long-running and confined to high-memory hardware. Collapsing these into a single class discards exactly the heterogeneity that determines how much flexibility the system can offer and at what cost.

This paper models that heterogeneity explicitly, extending a population-level EV charging model [6] to the

GPU data center setting with three additions: a fleet partitioned into memory classes; four job types with distinct execution and deadline distributions; and an admission layer that assigns queued jobs subject to memory feasibility and a power-headroom budget. Control is a myopic single-slot LP whose advantage lies in the granularity of hardware constraints it resolves, not in temporal foresight. The λ -sweep and transition matrix analysis reveal that the bilateral control window is empty under this policy, a finding that is itself a contribution, and that provides the analytical foundation for the MPC extension presented as a comparison section.

The paper is organized as follows. Section 2 presents the system model and LP controller. Section 3 characterizes stability via the λ -sweep and transition matrix. Section 4 presents results and discussion. Section 5 concludes.

2 System Model and Control Policy

2.1 GPU Fleet, Frequency Levels, and Power

The simulated data center operates $N=1,000$ GPUs partitioned into three memory classes $\mathcal{G} = \{\text{A16, A40, A80}\}$:

Class	N_g	M_g (GB)	Primary job types
A16	333	16	Prefill
A40	333	40	Decode, Fine-tuning
A80	334	80	All (training exclusive)

Each GPU operates at one of $K=5$ discrete DVFS levels with associated compute rates r_k and power draws $P(f_k)$:

Level k	Rate r_k	Power $P(f_k)$
0 (idle)	0.00	50 W
1	0.25	150 W
2	0.50	200 W
3	0.75	250 W
4 (full)	1.00	300 W

The frequency and power values above were chosen to reflect the behavior of real data center GPUs. According to the NVIDIA A100 datasheet [4], the A100 PCIe has a thermal design power of 300 W at full load, which we adopt as $P(f_4)$. The idle power of 50 W is consistent with empirical measurements of GPU leakage and context-retention power in cloud clusters reported by Patel et al. [5], who observed idle-but-allocated GPUs drawing roughly 50 W above their hardware minimum.

Total data center power is:

$$P_{\text{DC}}(t) = \sum_g n_g^{\text{idle}}(t) P_{\text{idle}} + \sum_{g, \tau, s, k} u_{\tau, s, k}(t) n_{\tau, s, g}(t) P(f_k), \quad (1)$$

establishing an idle power floor of 50 kW. The controllable flexibility bandwidth, meaning the power range achievable by throttling all non-deadline-critical GPUs between minimum and maximum frequency, is:

$$R_{\text{flex}}(t) = \sum_g \sum_{\tau, s: s>0} n_{\tau, s, g}(t) [P(f_K) - P(f_1)]. \quad (2)$$

2.2 State Space and Population Dynamics

Each active GPU carries a state (τ, s, g) : residual work τ , deadline slack s , and memory class g . The quantization grid $Q=4$ is required to close state transitions under fractional rates $r_k \in \{0.25, 0.5, 0.75, 1.0\}$. The class g is part of the state key so that completing jobs return

GPUs to the exact class that ran them, preserving the per-class conservation identity $n_g^{\text{idle}} + \sum_{\tau, s} n_{\tau, s, g} = N_g$. The population evolves as:

$$n_{\tau, s, g}(t+1) = a_{\tau, s, g}(t+1) + \sum_{k=1}^K u_{\tau', s', k}(t) n_{\tau', s', g}(t), \quad (3)$$

where $(\tau', s') = (\tau + r_k, s + 1 - r_k)$ is the predecessor state at rate k and $a_{\tau, s, g}$ are newly admitted jobs. States with $s=0$ are pinned to $k=K$ before the LP is solved to guarantee deadline satisfaction.

2.3 Job Types and Arrivals

Arrivals follow a Poisson process at rate λ , split uniformly across $J=4$ types. Residual time τ_r and slack τ_s are drawn from geometric distributions, clipped to $(\tau_{\text{max}}, s_{\text{max}}) = (50, 20)$. The job type parameters, grounded in published LLM inference characterizations [1, 2, 8], are listed in Table 1.

Table 1: Job type parameters. $\bar{\tau}_r$ = mean residual time (slots), $\bar{\tau}_s$ = mean slack (slots), μ = memory requirement (GB).

Type	$\bar{\tau}_r$	$\bar{\tau}_s$	μ	Notes
Prefill	2	1	16	Compute-bound, tight deadline
Decode	30	20	40	Memory-bound, deferrable
Fine-tuning	15	10	40	Intermediate flexibility
Training	40	25	80	A80 exclusive, longest

Each arriving job is queued under the four-tuple $(\tau_r, \tau_s, \mu, \text{type})$ so that deadline misses can be decomposed by both memory class and job type.

2.4 LP Controller and Admission

At each slot, a myopic single-slot LP determines frequency fractions $u_{\tau, s, k}$ for all free states ($s>0$), minimizing power deviation:

$$\min_{u, e, d} e + \lambda_{\text{smooth}} \sum_{(\tau, s, k)} d_{\tau, s, k}, \quad (4)$$

subject to:

$$\pm \left(\sum_{\tau, s, k} n_{\tau, s} u_{\tau, s, k} P(f_k) - P_{\text{busy}}^* \right) \leq e, \quad (5)$$

$$\sum_k u_{\tau, s, k} = 1 \quad \forall (\tau, s) \in \mathcal{F}, \quad (6)$$

$$\pm (u_{\tau, s, k} - u_{\tau, s, k}(t-1)) \leq d_{\tau, s, k}, \quad (7)$$

where $P_{\text{busy}}^* = \max(0, P_{\text{target}} - P_{\text{idle}} - P_{\text{fixed}})$ is the residual budget after subtracting idle and fixed-state power. The LP is solved via HiGHS with sparse CSC

constraint matrices. The smoothing weight $\lambda_{\text{smooth}}=100$ was selected empirically.

Admission follows the frequency solve. The controller estimates power freed by completing jobs this slot:

$$\hat{P}_{\text{freed}}(t) = \sum_{(\tau, s, g)} \sum_{k: \tau - r_k \leq 0} u_{\tau, s, k} n_{\tau, s, g} [P(f_k) - P_{\text{idle}}], \quad (8)$$

and admits jobs from the queue in Earliest Deadline First (EDF) order up to the headroom $H(t) = (P_{\text{target}} - P_{\text{DC}}) + \hat{P}_{\text{freed}}$, subject to idle GPU availability per class. EDF minimizes deadline misses under work-conserving scheduling [3]; a comparison against the slack-ratio alternative confirmed its superiority and is discussed in Section 4.

3 Stability and Capacity Analysis

3.1 Analytical Capacity Bound

The maximum number of GPUs that can run at full frequency within 150 kW is:

$$N_{\text{active}}^* = \frac{P_{\text{target}} - NP_{\text{idle}}}{P(f_K) - P_{\text{idle}}} = \frac{150,000 - 50,000}{250} = 400. \quad (9)$$

With mean residual time $\mathbb{E}[\tau_r] = (2+30+15+40)/4 = 21.75$ slots, the back-of-envelope stability threshold is:

$$\lambda^* = \frac{N_{\text{active}}^*}{\mathbb{E}[\tau_r]} = \frac{400}{21.75} \approx 18.4 \text{ jobs/slot}. \quad (10)$$

3.2 Lambda-Sweep Results

To empirically characterize the operating envelope, simulations were run across $\lambda \in \{5, 8, 10, 12, 16, 18, 20, 25, 40, 60\}$ jobs/slot with all other parameters held fixed. Each run covers $T=1,000$ slots with the first 100 discarded as warmup. The results in Figure 1 and Figure 2, along with Tables 3 and 4, tell a consistent story across three distinct regimes.

For $\lambda \leq 20$, the queue stays bounded and miss rates are effectively zero, but average power falls well short of 150 kW (only 109.4 kW at $\lambda=18$). Above $\lambda=25$, the queue begins diverging sharply while power finally approaches target. No single arrival rate achieves both a bounded queue and accurate power tracking simultaneously, which is the central finding discussed in Section 4.2. The per-type miss rate decomposition in Figure 2 shows that training jobs drive most losses, followed by decode, while prefill remains near-zero throughout given A16’s large capacity margin.

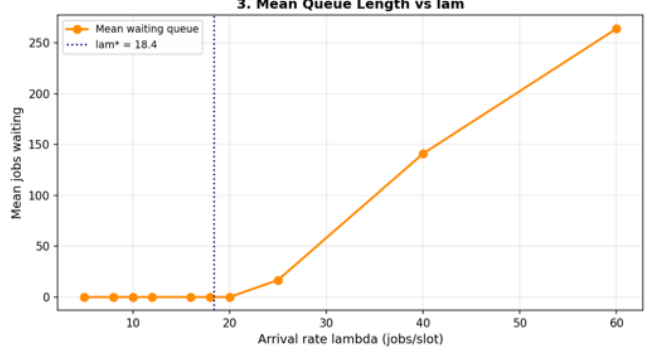


Figure 1: Mean queue length vs. λ under the myopic LP (flat target). The queue stays near zero below $\lambda^* \approx 18.4$ and diverges sharply above it, with 141 jobs waiting at $\lambda=40$ and 264 at $\lambda=60$.

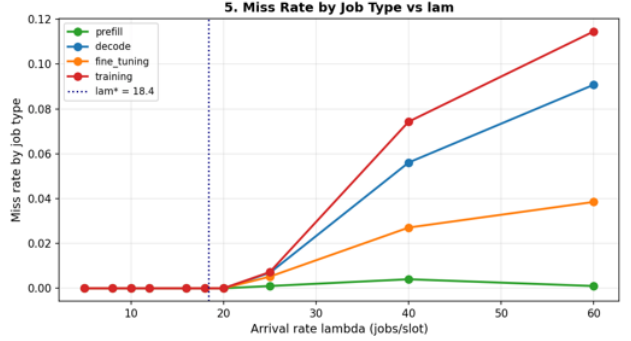


Figure 2: Miss rate by job type vs. λ . Training (A80-exclusive) and decode (A40-dominant) drive most losses. Prefill remains near zero throughout, reflecting the A16 pool’s large capacity margin.

Summary Table

lam	Miss %	RMS kW	Avg Queue	Avg Power kW
5	0.0%	92.5	0	57.6
8	0.0%	82.0	0	68.1
10	0.0%	74.5	0	75.7
12	0.0%	66.6	0	83.7
16	0.0%	49.7	0	101.2
18	0.0%	41.9	0	109.4
20	0.0%	33.2	0	118.9
25	2.0%	14.8	17	144.9
40	16.2%	12.4	141	147.7
60	24.5%	9.7	264	149.4

Figure 3: Summary table for the flat power target sweep.

Summary Table

lam	Miss %	RMS kW	Avg Queue	Avg Power kW
5	0.0%	95.0	0	57.6
8	0.0%	84.8	0	68.2
10	0.0%	78.0	0	75.7
12	0.0%	70.3	0	83.7
16	0.0%	53.6	0	101.9
20	0.9%	36.6	4	120.9
25	5.8%	21.4	31	138.3
40	19.5%	15.1	147	144.4
60	27.4%	12.4	282	145.7

Figure 4: Summary table for the sinusoidal power target sweep. Results are consistent with the flat case: the same three regimes appear and λ^* is governed by the mean target, not the instantaneous value.

3.3 Operation Under Chronic Overload

Before the λ -sweep study, an earlier set of parameter tuning experiments was conducted at $\lambda \in \{60, 70, 80\}$, all well above $\lambda^*=26.54$. In this regime the queue diverges and miss rates cluster between 32 and 42% regardless of parameter choices, because the system is shedding deadline overflow rather than exhibiting controller behavior. These results are reported here for completeness and should be read as characterizations of loss-system behavior, not as evidence of controller performance.

An earlier urgency metric $s/(\tau+s)$ was also tested. It consistently produced miss rates of 64 to 72% by inflating priority for low-slack jobs relative to their remaining work, causing the scheduler to over-commit high-memory classes. All subsequent experiments use $(\tau+s)$ ascending (Earliest Deadline First), which reduced miss rates to the 32 to 42% range in the overloaded regime.

3.4 Transition Matrix and Per-Class Bounds

To move beyond the scalar bound of Eq. (10), we built a per-class substochastic transition matrix $\mathbf{A}_g$ and computed its spectral radius and mean service time. The

Table 2: Flat power curve under chronic overload ($\lambda \gg \lambda^*$), urgency $\tau+s$. Miss rates reflect queue overflow, not controller performance.

λ	Arrivals	Missed	Miss %	Util. %	Avg. Power
80	80,016	33,172	41.5	63.7	151.89 kW
70	70,271	26,345	37.5	63.2	150.73 kW
60	59,998	19,389	32.3	63.0	150.0 kW

Table 3: Sinusoidal curve (150kW midpoint) under chronic overload.

λ	Arrivals	Missed	Miss %	Util. %	Avg. Power
60	59,998	20,403	34.0	61.8	148.0 kW
70	70,271	26,979	38.4	63.3	150.9 kW
80	80,016	33,685	42.1	63.7	152.1 kW

full construction follows [7]; the key quantities are the spectral radius $\rho(\mathbf{A}_g)$, which confirms the policy is inherently draining ($\rho < 1$ for all classes), and the mean service time:

$$\mathbb{E}[\text{svc}]_g = \mathbf{1}^\top (\mathbf{I} - \mathbf{A}_g)^{-1} \mathbf{p}_g, \quad (11)$$

where $\mathbf{p}_g$ is the initial state distribution of a job routed to class g . The per-class stability threshold in terms of total arrival rate is:

$$\lambda_{g,\text{total}}^* = \frac{N_g}{\mathbb{E}[\text{svc}]_g \cdot \phi_g}, \quad (12)$$

where $\phi_g = |\mathcal{J}_g|/J$ is the arrival fraction routed to class g . Table 4 summarizes the results.

Table 4: Per-class transition matrix results.

Class	ρ	$\mathbb{E}[\text{svc}]$	ϕ_g	λ_{class}^*	λ_{total}^*
A16	0.41	3.58	0.25	93.13	372.51
A40	0.40	25.09	0.50	13.27	26.54
A80	0.41	32.98	0.25	10.13	40.51
System λ^* (binding class: A40)					26.54

4 Results and Discussion

4.1 Power Tracking Performance

Figures 5 and 6 show the power consumption time series for the flat and sinusoidal targets respectively, both at $\lambda=60$. In the flat case, the controller ramps up from the idle floor of roughly 50 kW during the first 100 slots as jobs accumulate and then settles into steady-state tracking around 150 kW with small fluctuations. The LP successfully follows the target throughout, though

the oscillatory pattern visible in steady state reflects the limit cycle discussed below.

The sinusoidal case tells a similar story but with a more demanding target. The controller tracks the 150 ± 30 kW sinusoid reasonably well once the system warms up, with the actual power curve following the shape of the target across multiple periods. There are visible deviations near the troughs where the controller struggles to reduce consumption fast enough, which is consistent with the one-sided undershoot failure mode: at low P_{target} phases the fleet still has jobs running that were admitted during high-target phases and cannot be immediately throttled below their forced minimum. Both profiles confirm that the myopic LP does achieve load following in a general sense, but the behavior is imperfect and its limitations become clearer when viewed alongside the queue and stability results.

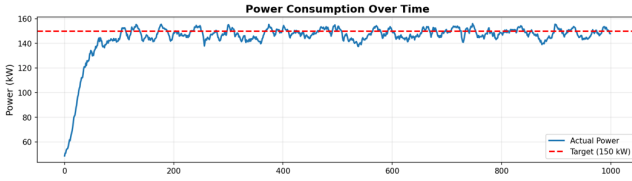


Figure 5: Power consumption over time for the flat 150kW target at $\lambda=60$. The system ramps up from the 50kW idle floor and tracks the target in steady state with small oscillations.

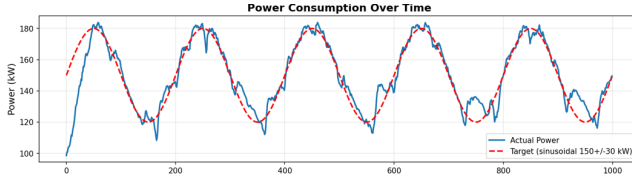


Figure 6: Power consumption over time for the sinusoidal target 150 ± 30 kW at $\lambda=60$. The LP tracks the sinusoid across multiple periods, with visible undershoot near the troughs.

The steady-state oscillations in both figures are a direct consequence of the myopic horizon. When a cohort of jobs completes simultaneously, the power headroom spikes, the admission controller floods the fleet with queued jobs in a single slot, and the LP then throttles all newly admitted jobs aggressively to bring power back to target, which extends their service times and locks GPUs for longer than expected. The queue rebuilds, the next cohort completes, and the cycle repeats. This is not a tuning artifact: the dedicated λ_{smooth} sweep confirmed that the oscillation persists even at large smoothing weights, pointing to a structural limitation of single-slot optimization rather than a controller deficiency.

4.2 The Bilateral Control Window

The central finding from Section 3.2 is that no bilateral control window exists under the myopic LP. The controller needs approximately $\lambda=60$ to approach 150kW average power, but at that point the queue already holds 264 jobs and the miss rate has reached 24.5%. The system is always in one of two failure modes: stable with a bounded queue but power well below target ($\lambda \leq 20$, average power ≤ 118.9 kW), or overloaded with accurate power tracking but a diverging queue and growing deadline losses ($\lambda \geq 40$).

This is a structural consequence of the finite GPU pool and single-slot optimization rather than a tuning failure. The A40 class is the binding constraint with $\lambda^*=26.54$ jobs/slot, as established in Section 3.4. A40 absorbs 50% of all arrivals because both decode and fine-tuning each represent 25% of the uniform split and both compete for the same 333 GPUs. Even at $\lambda=60$, the A16 pool remains largely idle while A40 and A80 saturate, reflecting the structural underutilization of A16 capacity under finite-pool constraints. This also motivates an infinite-pool variant in which admission is gated purely by power headroom and the control problem reduces to a pure power-tracking problem without class-level bottlenecks.

4.3 Motivation for MPC

The empty bilateral control window is the strongest case for MPC that the myopic LP results produce. By optimizing jointly over a horizon of H future slots, MPC can anticipate the completion-admission-throttle cycle that drives the oscillations described above, smooth frequency trajectories across time, and expand the effective bilateral control window. Under MPC, the smoothing penalty λ_{smooth} is no longer needed since the horizon itself provides implicit smoothing by penalizing any frequency assignment that causes a tracking violation in a future slot. The follow-on section of this report will present MPC as a focused comparison, evaluating whether the horizon controller closes the bilateral control gap that the myopic LP cannot.

5 Conclusion

This paper presented a population-based simulation framework for GPU data center demand response with heterogeneous workloads and memory classes. The myopic LP controller, characterized through a λ -sweep and transition matrix analysis, reveals an empty bilateral control window: no arrival rate simultaneously achieves bounded queue and full power-target tracking under the finite GPU pool. The A40 class is identified as the binding constraint at $\lambda^*=26.54$ jobs/slot, correctly predicted by the per-class transition matrix and confirmed

empirically. These findings provide the analytical foundation for the MPC extension, which will be presented as a comparison section evaluating whether a horizon controller closes the gap the myopic LP cannot.

References

- [1] Murilo Argerich et al. Characterizing LLM inference energy-performance tradeoffs across workloads and GPU scaling. *arXiv preprint arXiv:2501.08219*, 2025.
- [2] Abhimanyu R. Bambhaniya et al. Reducing compute waste in LLMs through kernel-level DVFS. *arXiv preprint arXiv:2601.08539*, 2026.
- [3] C. L. Liu and James W. Layland. Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM*, 20(1):46–61, 1973.
- [4] NVIDIA Corporation. *NVIDIA A100 Tensor Core GPU Datasheet*, 2022.
- [5] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Riccardo Bianchini. Towards improved power management in cloud GPUs. In *IEEE Computer Architecture Letters*, 2023.
- [6] Anna Scaglione and Udit Gupta. Technical note: GPU data center demand response framework. 2025. Internal technical note.
- [7] Anna Scaglione and Udit Gupta. Next steps for the GPU data center simulation framework. April 2026. Technical note.
- [8] Jovan Stojkovic et al. A systematic characterization of LLM inference on GPUs. *arXiv preprint arXiv:2512.01644*, 2025.